

# Did the AI Really *Need* the Rulebook, or Did It Already Know the Answer?

A plain-language companion to the paper

*This is a plain-language companion to the technical paper, written so the argument can be followed end to end without the formalism. It is a companion, not the paper of record; every claim here is stated more carefully (and hedged) in `main.tex`. It is a **test bench**—a way to check the machinery in simulation—not a proof that real students learn more.*

## The problem in one paragraph

Picture an AI that is supposed to answer using an approved rulebook or reference corpus—say, the rules that stop ships from colliding, or a set of facts about your company’s products. It gives a right-looking answer. Here is the question almost nobody can answer: **did it actually need the corpus?** Maybe it read your rule and used it. Or maybe it already knew the answer from its own training and your corpus added nothing—in which case the moment your rules change, or the questions shift, the whole thing quietly breaks and you won’t see it coming. That is the **necessity** question. There is a twin question: is the corpus **sufficient**—does it actually contain what’s needed to do the job? Normally you’d find out by running an expensive study with real people. We wanted a cheap way to answer both questions *before* involving anyone.

## Idea 1: a test with a right answer (the “Transfer Instrument”)

Multiple-choice quizzes are a weak way to measure this, because a fluent-sounding answer can still be wrong in practice. So instead we drop the learner into a **realistic task that has a measurable best move**: steering a ship to avoid a collision. For any situation, a standard piece of autopilot software computes a strong “reference” move. We then score the learner by **how far its decision lands from that reference move**. Small distance = good; large distance = bad. We call that distance *regret*. Unlike a quiz score, regret is smooth—it can tell “slightly off” from “about to crash.”

We also build the rulebook so that **each rule controls exactly one part of the score**. Remove the “turn to starboard” rule and *only* the “did you turn the right way” part of the score gets worse; nothing else moves. We check this actually holds—it’s not automatic. That “one rule → one number” property is what lets us test rules one at a time.

## Idea 2: run the test backwards to measure NECESSITY

Here is the core move: **take a corpus item out and re-run the test**. If the score gets worse, the learner was genuinely relying on that item—it *needed* it. If the score *doesn’t budge*, the learner

already knew the answer and never needed your corpus. That gap—how much performance falls when we remove an item—*is* the necessity of that item. Same instrument that grades learning, run in reverse, now audits the corpus.

## The headline: proving necessity is real, by *building* the ground truth

Here’s the honest soft spot, and how we close it—this is the biggest change in the whole project. When we remove an item and the score doesn’t move, we *conclude* the model already knew the answer. But how do we *know* it already knew it? We could be fooling ourselves. The only airtight way to prove necessity is real, and not confounded by what the model happened to already know, is to **control what the model knows**—and watch necessity move in lockstep.

So we do exactly that: we **teach a fact directly into a model’s weights** and watch its necessity for that fact fall. Before teaching, the model must read the corpus to answer, so necessity is high. After teaching, it knows the fact on its own, so necessity should drop to near zero. If it does, the instrument is measuring what we claim.

**The calibration (fictional-facts Q&A, NO simulator—just an answer checker).** No ship, no autopilot, no hand-built barrier: just made-up facts about fictional places that no model could possibly know, and a plain checker that asks “is the answer right?” We teach those facts into real models, a bit at a time, and necessity falls **smoothly and steadily toward zero** as the model soaks the facts up—smooth *because there are many independent facts* (learn a quarter, then half, then almost all). We ran it in *two* different model families (Qwen and Phi), three seeds each, and both give the same clean “dose-response” curve—sliding from fully corpus-reliant to almost self-sufficient, differing only in exactly where the drop-off sits. That two unrelated models agree is the evidence it isn’t a quirk of one. It uses no simulator, so the method isn’t a lucky trick of one hand-built environment; it works on any task with a checkable right answer.

**An honest twist we found (knowing vs. doing).** We also tried the same teaching trick on a *decision* instead of a fact—a ship that must turn to avoid a hidden hazard. Here something surprising happened, and we report it straight because it’s important. After teaching, the model can *say* the fact perfectly (ask it and it tells you to turn—recall jumps from 0 to 0.78), but its *actual steering decision doesn’t budge*: it still grounds. It *knows* but doesn’t *do*. The gap closes only when we make the model *reason first* (“is there a hazard here, and what should I do?”), which pulls the buried knowledge into the decision. And trying to force the behavior by training on the decisions directly *backfires*—the model starts “seeing” the hazard everywhere and turning blindly, rather than learning the specific fact. The lesson: whether a fact “counts” as necessary depends on whether you measure what the learner *knows* or what it *does*, and those can come apart. For a tutor (where the point is what the learner knows), the fact-recall measure is the right one; for an autopilot (where only the action matters), the decision measure is, and a fact that’s known-but-not-used is a real failure. The instrument has to say which one it’s measuring.

## What this is *for*: two products

**Product 1 — Corpus diagnosis.** Run the test backwards over every rule or fact and **rank them by how much the learner relies on each one**. That already tells you which items are carrying weight and which are dead weight. The sharp part is splitting “the corpus didn’t help”

into two *opposite* cases that look identical from the outside:

- **Redundant** — the model already knew it, so removing it changes nothing. *Verdict: delete it, it's clutter.*
- **Unusable** — the model fails the task *even with the item right in front of it*. The item has value; this model just can't act on it. *Verdict: the problem is the model, not the corpus.*

Removing-and-watching (necessity alone) can't tell these apart—both show “no change.” What separates them is the score *with* the item present: near-perfect  $\Rightarrow$  redundant; still failing  $\Rightarrow$  unusable. We saw both live on a suite of eight hidden hazards: Claude Opus relies on *all eight* (it reads the corpus and clears), while Llama-4-Maverick grounds even with the rule right in front of it on the reaches it should override—the textbook “unusable” case (fix the model, not the corpus).

**Product 2 — Corpus sufficiency, and the FALSE SUFFICIENCY alarm.** This is the diagnostic ordinary RAG scoring simply cannot produce. Imagine a corpus that looks totally sufficient: the model answers everything correctly, every check is green. Our instrument can tell you the accuracy is a mirage—the model is **coasting on what it already knew**, and the corpus is contributing nothing (necessity  $\approx 0$  everywhere). We call that **false sufficiency**, and it's a warning: this system will fail the instant the questions drift outside what the model happens to have memorized. *Honest caveat:* sufficiency is always “sufficient for *these* questions.” We can only test the questions we ask, so we always report how many we asked—and the false-sufficiency alarm is precisely the measurable warning that a looks-fine-on-this-question-set corpus is fragile.

## How this differs from existing tools (RAGAS, ContextCite)

Tools like RAGAS and ContextCite are good, and close by, so it's worth being precise. They ask whether an answer is **supported by** or **influenced by** the provided context—did these words come from that passage? We ask a different question: did the model **need** the context at all? And crucially, we **calibrate** that against a known baseline by teaching facts in ourselves, so a fact the model already knew is never miscounted as one it relied on. That calibration—controlling what's in the weights and checking necessity tracks it—is the thing those tools structurally lack. It's the difference between “the answer matches the passage” and “the answer would have been wrong without the passage.”

## Additional breadth (clearly separate from the central story)

Everything above is the **central story**: measure necessity, prove it's real by construction (the fictional-facts calibration), flag the honest knowing-vs-doing caveat, and turn the measure into diagnosis and sufficiency products. The following are supporting breadth—they widen the evidence, but the paper does not lean on them.

- **Cross-model discrimination.** We run the instrument across five production LLMs (on AWS Bedrock) and it tells them apart on a *graded* scale: every model reads the *textbook* ship rules as redundant (they all already know the COLREGs), but on a suite of eight *invented* hidden hazards reliance falls cleanly—Claude Opus relies on all eight, Haiku on four, Sonnet on three, Nova-micro on two, and Llama-4-Maverick on none (it grounds even with the rule present). Same textbook knowledge, genuinely different reliance on the corpus-only facts—exactly the dynamic range the standard rulebook can't show.

- **Lookup vs. reasoning (reading depth).** Beyond “did the learner use the corpus at all,” the same instrument separates a learner that *looks a rule up* from one that *reasons with it*: on cases where a local rule *overrides* the usual “turn starboard” reflex, a lookup-only learner grounds while a reasoner clears. Run live, Gemini-2.5-flash reads as a *reasoner*—it relies on the local override rule every time (5/5) on the cases where the reflex would fail.
- **A second simulator domain.** Besides the continuous steering task, we built a *discrete, step-by-step* task—a roadside tire change—and showed the same machinery works there too (each skill controls exactly one part of the score, and competence and performance move together). So the “one rule, one number” property isn’t special to ships.
- **The unlearning “says  $\neq$  does” sidelight.** An earlier version of this project made machine-*unlearning*—surgically erasing a rule from the model’s brain—the load-bearing proof. It isn’t anymore; the teaching-in dose-response above is the primary weight-level validation. But the unlearning result is a lovely illustration in its own right: when we *gently* erase “turn to starboard,” the model stops *saying* the rule (it quits citing “Rule 14,” and every text-based “did it forget?” score reports success—probe dropped from 0.43 to 0.27), yet on the actual steering test the ship **still turns to starboard, unchanged** ( $\Delta = 0.000$ ). The model changed what it *says*, not what it *does*—and our instrument measures the *decision*, so it wasn’t fooled. A supporting sidelight, not the keystone.
- **Honest limitations.** (1) Sufficiency is only ever “sufficient for the questions we asked”—a bounded question set (see false sufficiency above). (2) Everything here is **simulation and mechanism evidence, not a human study**. A real trial with real learners is written down as the next step, not skipped. (3) We designed our rulebooks so each rule controls one thing; real corpora are messier, and when rules overlap the diagnosis gets coarser (it points at a *group* of items, not one).

## What we are *not* claiming (the honest part)

- **This is a test bench, not proof that students learn more.** Everything is simulation. The real human study is the next step, not a result in hand.
- **We are not claiming a new AI tutor, or a new collision-avoidance method.** We borrow mature ship-autopilot software as a *measuring stick*. The measurement method is the contribution.
- **We are not claiming the corpus is “sufficient” in general**—only for the specific questions we probed. The false-sufficiency alarm is our honest handle on that limit.